

We define static methods by placing the `@staticmethod` decorator before their `def` statements. Here is an example of a static method.

```
class ExampleClassWithStaticMethod:
    @staticmethod
    def sayHello():
        print('Hello!')

# Note that no object is created, the class name precedes sayHello():
ExampleClassWithStaticMethod.sayHello()
```

There would be almost no difference between the `sayHello()` static method in the `ExampleClassWithStaticMethod` class and a `sayHello()` function. In fact, you might prefer to use a function, because you can call it without having to enter the class name beforehand.

Static methods are more common in other languages that don't have Python's flexible language features. Python's inclusion of static methods imitates the features of other languages but doesn't offer much practical value.

When to Use Class and Static Object-Oriented Features

You'll rarely need class methods, class attributes, and static methods. They're also prone to overuse. If you're thinking, "Why can't I just use a function or global variable instead?" this is a hint that you probably don't need to use a class method, class attribute, or static method. The only reason this intermediate-level book covers them is so you can recognize them when you encounter them in code, but I'm not encouraging you to use them. They can be useful if you're creating your own framework with an elaborate family of classes that are, in turn, expected to be subclassed by programmers using the framework. But you most likely won't need them when you're writing straightforward Python applications.

For more discussion on these features and why you do or don't need them, read Phillip J. Eby's post "Python Is Not Java" at <https://dirtsimple.org/2004/12/python-is-not-java.html> and Ryan Tomayko's